

ANOTAÇÕES

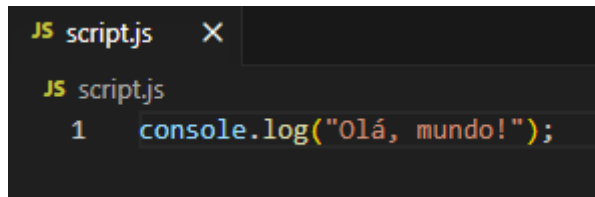
Período Trainee 2026

**Trilha 03 - Desenvolvimento backend e
APIs simples**

**Aula 05 - Introdução ao Node.js e ao
Express.js**

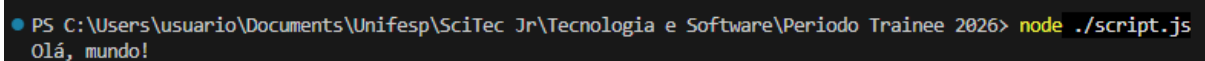
ANOTAÇÕES

- Node.js é um ambiente de execução JavaScript fora do navegador. É o que permite rodar JavaScript fora do navegador;
- Por exemplo:



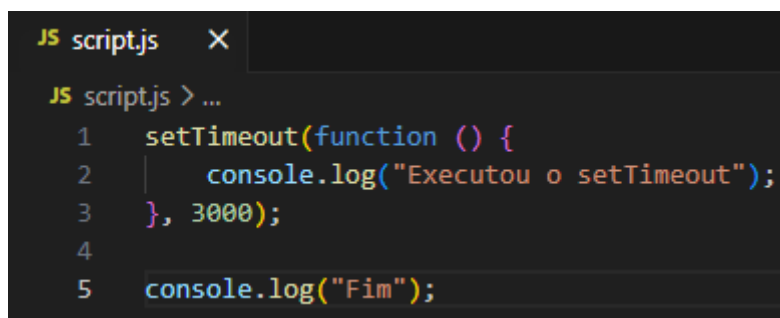
```
JS script.js X
JS script.js
1 console.log("Olá, mundo!");
```

- Para executar isso, eu preciso do software Node, que vai compilar o meu código:

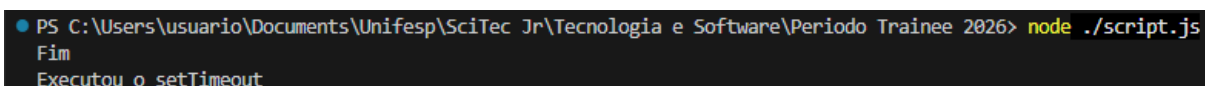


```
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026> node ./script.js
Olá, mundo!
```

- Permite criar servidores, APIs e usar ferramentas de desenvolvimento modernas;
- Permite programação assíncrona com eventos, por exemplo:



```
JS script.js X
JS script.js > ...
1 setTimeout(function () {
2 |   console.log("Executou o setTimeout");
3 | }, 3000);
4
5 console.log("Fim");
```



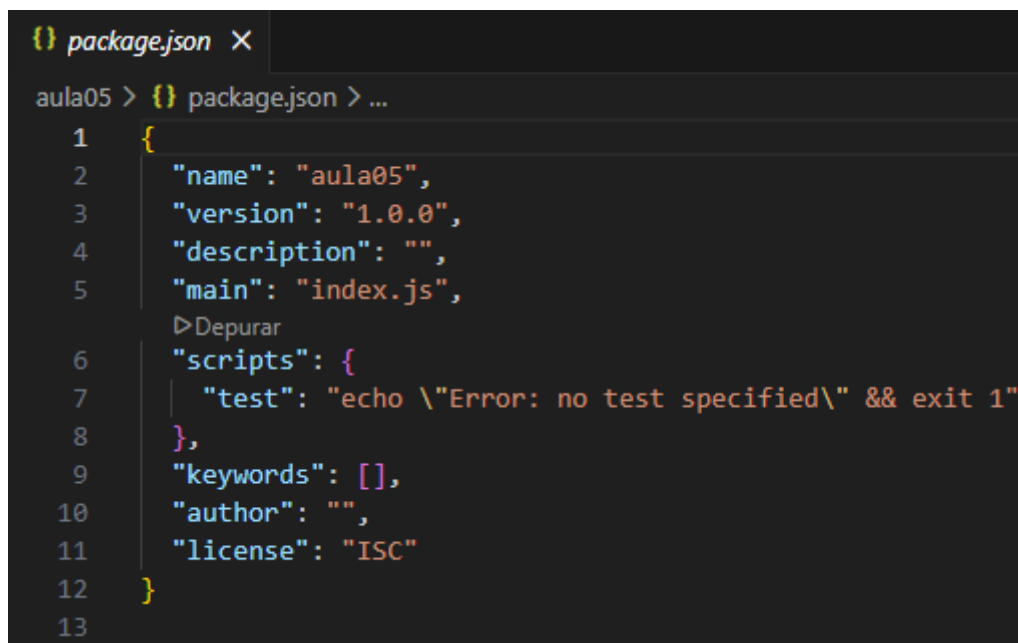
```
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026> node ./script.js
Fim
Executou o setTimeout
```

- Na programação síncrona, o código é executado na ordem das linhas de comando. Já na programação assíncrona, podemos deixar uma função “esperando” (delay de 3s), enquanto outras são executadas;
- Para testar o Node:

```
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026> node -v
v22.19.0
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026> npm -v
10.9.3
```

- Para criar um projeto em Node, precisamos primeiro estar na pasta que vamos utilizar e inicializamos o projeto da seguinte forma:

```
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026> cd aula05
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026\aula05> npm init -y
```



```
{} package.json X
aula05 > {} package.json > ...
1  {
2    "name": "aula05",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC"
12 }
13
```

- Esse arquivo contém o nome, a versão do projeto, a descrição e todas as configurações do projeto;
- (Criar o arquivo index.js na pasta do projeto);
- Nos scripts, podemos fazer o seguinte (trocar o *test*):

```

{} package.json X
aula05 > {} package.json > {} scripts > abc start
1  {
2    "name": "aula05",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start": "node index.js"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC"
12 }
13

```

- Assim, toda vez que a gente rodar “npm run” e o nome do script (no caso, start), ele executa o arquivo principal;
- (Votar para a apresentação);
- Express é um framework para Node.js. Utilizado para construção de APIs e servidores web. Ele simplifica criação de servidores, criação de rotas, tratamento de requisições, organização da aplicação;
- Vamos instalar o Express via terminal:

```

PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026\aula05> npm install express

added 66 packages, and audited 67 packages in 6s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

- Podemos observar que ele entrou nas dependências do projeto (fazer a inclusão do “type”: “module”,):

```
{} package.json X
aula05 > {} package.json > abc type
1  {
2    "name": "aula05",
3    "version": "1.0.0",
4    "description": "",
5    "type": "module",
6    "main": "index.js",
7    "scripts": {
8      "start": "node index.js"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "dependencies": {
14     "express": "^5.2.1"
15   }
16 }
17
```

- A pasta node_modules é onde está instalado todas as dependências que o Node utiliza;
- Vamos fazer um script básico de servidor:

```

JS index.js X
aula05 > JS index.js > ...
1  import express from "express";
2
3  const app = express()
4  const port = 3000;
5
6  /*
7   |   GET - Buscar
8   |   POST - Criar
9   |   PUT - Atualizar
10  |   DELETE - Remover
11  */
12
13  app.get("/", (req, res) => {
14  |   res.send("Olá, mundo!");
15  | });
16
17  app.listen(port, () => {
18  |   console.log(`Example app listening on port ${port}`);
19  | });

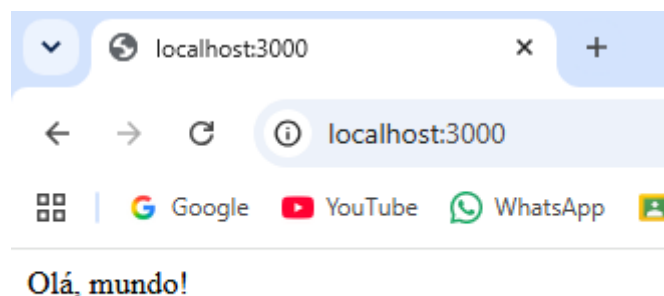
```

- Primeiro, importamos o pacote Express (que acabamos de instalar). Sem o Express, seria necessário utilizar os módulos nativos do Node.js, tornando o código mais complexo. Após essa linha, temos acesso à função `express()`;
- A função `express()` cria uma aplicação Express. Ela será responsável por receber requisições, processar rotas, enviar respostas, gerenciar middlewares. Podemos pensar em `app` como sendo o nosso servidor;
- Depois, definimos a porta. Todo servidor precisa estar associado a uma porta. Nesse caso, escolhemos a porta 3000;
- Esses são os principais métodos HTTP utilizados em APIs REST:
 - GET - Buscar informações;
 - POST - Criar novos registros;
 - PUT - Atualizar registros existentes;
 - DELETE - Remover registros;

- Com `app.get`, criamos nossa primeira rota. Nesse caso, quando alguém fizer uma requisição GET para `/`, a função a seguir é executada. `/` representa a rota principal da aplicação. Quando acessamos `http://localhost:3000`, estamos acessando a rota `/`, ou seja, a página inicial do servidor;
- A função recebe dois objetos:
 - `req` (request) - Representa a requisição enviada pelo cliente;
 - `res` (response) - Representa a resposta que será enviada ao cliente (podemos enviar textos, HTML, JSON, arquivos, códigos de status);
- Quando o usuário acessar `http://localhost:3000`, ele receberá "Olá, mundo!";
- A função `listen()` faz o servidor começar a escutar requisições. Enquanto essa linha não for executada, o servidor existe, mas ainda não está funcionando. O parâmetro `port` indica em qual porta o servidor ficará disponível;
- Callback: essa função é executada quando o servidor é iniciado com sucesso. No terminal aparecerá: `Example app listening on port 3000`;
- Vamos testar:

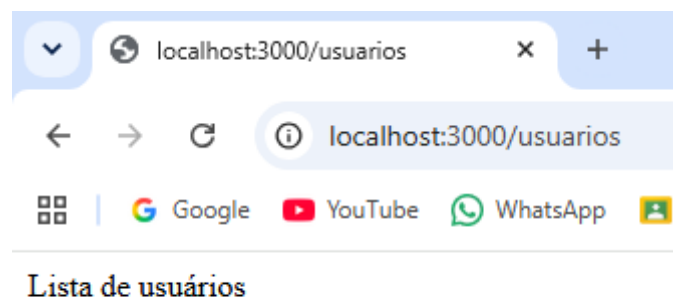
```
PS C:\Users\usuario\Documents\Unifesp\SciTec Jr\Tecnologia e Software\Periodo Trainee 2026\aula05> npm run start
> aula05@1.0.0 start
> node index.js

Example app listening on port 3000
```



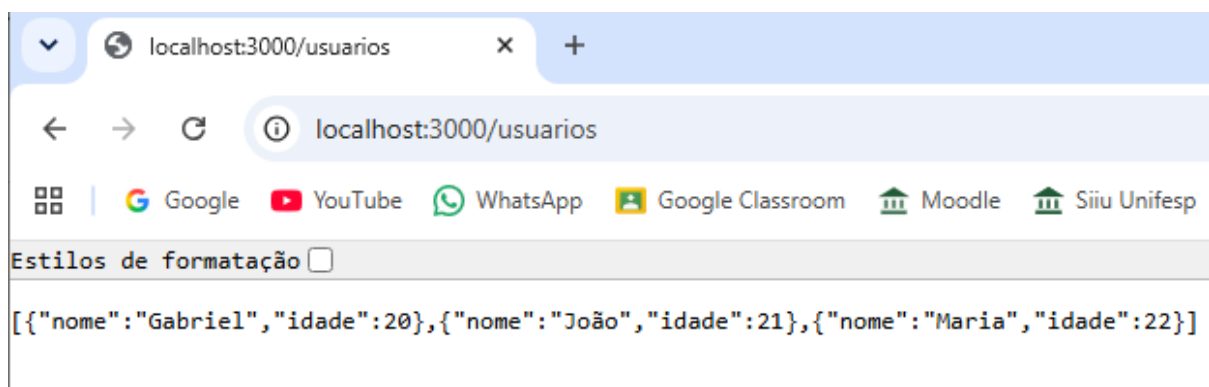
- Para encerrar, sempre utilizamos Ctrl + C no terminal;
- Podemos criar novas rotas:

```
JS index.js X
aula05 > JS index.js > ...
 1  import express from "express";
 2
 3  const app = express()
 4  const port = 3000;
 5
 6  /*
 7   GET - Buscar
 8   POST - Criar
 9   PUT - Atualizar
10   DELETE - Remover
11  */
12
13  app.get("/", (req, res) => {
14    res.send("Olá, mundo!");
15  });
16
17  app.get("/usuarios", (req, res) => {
18    res.send("Lista de usuários");
19  });
20
21  app.listen(port, () => {
22    console.log(`Example app listening on port ${port}`);
23  });
```



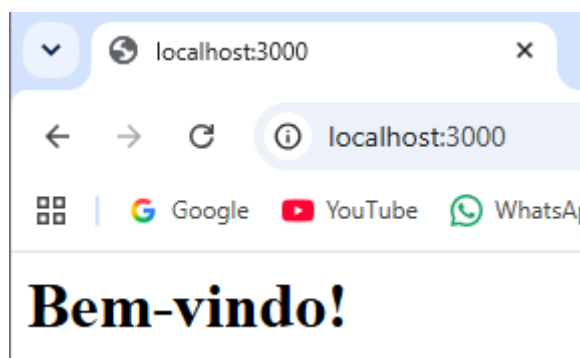
- Vamos simular uma lista de usuários para serem retornados em formato JSON (formato típico de dados):

```
JS index.js X
aula05 > JS index.js > ...
1  import express from "express";
2
3  const app = express()
4  const port = 3000;
5
6  /*
7   GET - Buscar
8   POST - Criar
9   PUT - Atualizar
10  DELETE - Remover
11 */
12
13 app.get("/", (req, res) => {
14   res.send("Olá, mundo!");
15 });
16
17 app.get("/usuarios", (req, res) => {
18   res.json([
19     { nome: "Gabriel", idade: 20 },
20     { nome: "João", idade: 21 },
21     { nome: "Maria", idade: 22 },
22   ]);
23 });
24
25 app.listen(port, () => {
26   console.log(`Example app listening on port ${port}`);
27 });
```



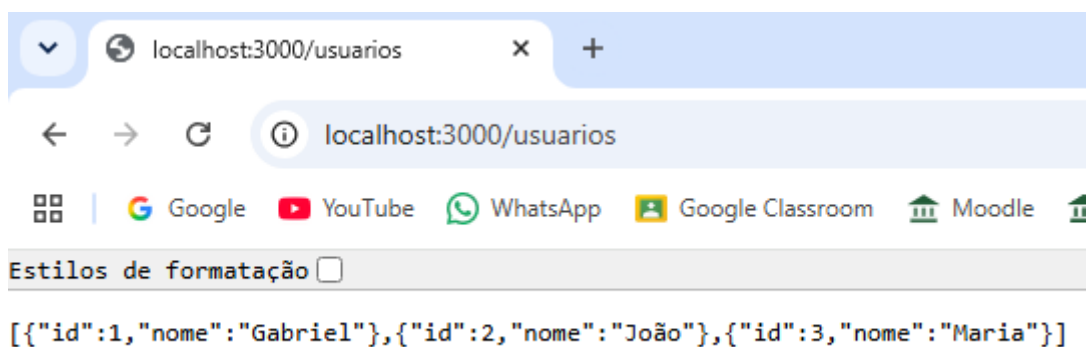
- Podemos retornar também um código HTML:

```
JS index.js X
aula05 > JS index.js > app.get("/") callback
1  import express from "express";
2
3  const app = express()
4  const port = 3000;
5
6  /*
7   GET - Buscar
8   POST - Criar
9   PUT - Atualizar
10  DELETE - Remover
11 */
12
13 app.get("/", (req, res) => {
14   res.send("<h1>Bem-vindo!</h1>");
15 });
16
17 app.get("/usuarios", (req, res) => {
18   res.json([
19     { nome: "Gabriel", idade: 20 },
20     { nome: "João", idade: 21 },
21     { nome: "Maria", idade: 22 },
22   ]);
23 });
24
25 app.listen(port, () => {
26   console.log(`Example app listening on port ${port}`);
27 });
```



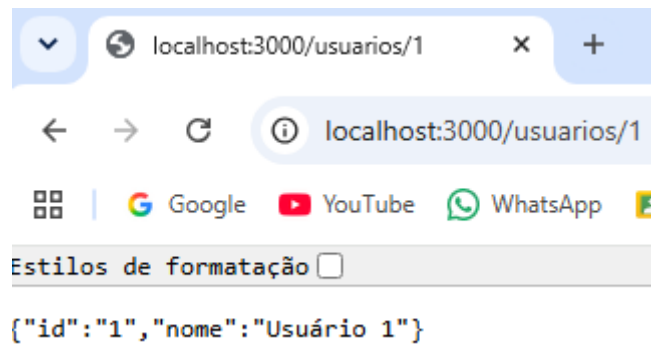
- Vamos modificar um pouco a nossa lista de usuários para explicar três conceitos: retornando JSON, parâmetros de rota e query strings;

```
JS index.js X
aula05 > JS index.js > app.get("/usuarios") callback
1  import express from "express";
2
3  const app = express()
4  const port = 3000;
5
6  /*
7   GET - Buscar
8   POST - Criar
9   PUT - Atualizar
10  DELETE - Remover
11 */
12
13 app.get("/", (req, res) => {
14   res.send("<h1>Bem-vindo!</h1>");
15 });
16
17 app.get("/usuarios", (req, res) => {
18   res.json([
19     { id: 1, nome: "Gabriel" },
20     { id: 2, nome: "João" },
21     { id: 3, nome: "Maria" },
22   ]);
23 });
24
25 app.listen(port, () => {
26   console.log(`Example app listening on port ${port}`);
27 });
```



```
JS index.js X
aula05 > JS index.js > ...
1  import express from "express";
2
3  const app = express()
4  const port = 3000;
5
6  /*
7   GET - Buscar
8   POST - Criar
9   PUT - Atualizar
10  DELETE - Remover
11 */
12
13 app.get("/", (req, res) => {
14   res.send("<h1>Bem-vindo!</h1>");
15 });
16
17 app.get("/usuarios", (req, res) => {
18   res.json([
19     { id: 1, nome: "Gabriel" },
20     { id: 2, nome: "João" },
21     { id: 3, nome: "Maria" },
22   ]);
23 });
24
25 app.get("/usuarios/:id", (req, res) => {
26
27   const { id } = req.params;
28
29   res.json({
30     id,
31     nome: `Usuário ${id}`
32   });
33
34 });
```

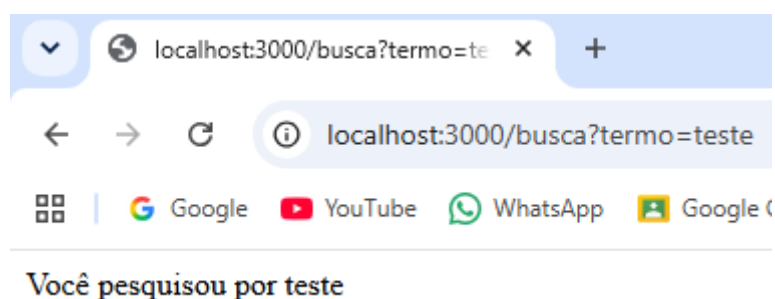
- O que significa :id? O :id é um parâmetro dinâmico. Ou seja, essa rota consegue atender vários endereços (/usuarios/1, /usuarios/2, /usuarios/10, /usuarios/500);



- req.params - Captura os parâmetros presentes na URL;
- Query String são informações enviadas após o símbolo ? na URL. Muito comum em sites convencionais, quando o usuário realiza alguma busca (como buscar um vídeo no YouTube);

```
JS index.js X
aula05 > JS index.js > ...
13 app.get("/", (req, res) => {
14   res.send("Olá, bem vindo!");
15 });
16
17 app.get("/usuarios", (req, res) => {
18   res.json([
19     { id: 1, nome: "Gabriel" },
20     { id: 2, nome: "João" },
21     { id: 3, nome: "Maria" },
22   ]);
23 });
24
25 app.get("/usuarios/:id", (req, res) => {
26
27   const { id } = req.params;
28
29   res.json({
30     id,
31     nome: `Usuário ${id}`
32   });
33 });
34
35
36 app.get("/busca", (req, res) => {
37
38   const { termo } = req.query;
39
40   res.send(`Você pesquisou por ${termo}`);
41
42 });
```

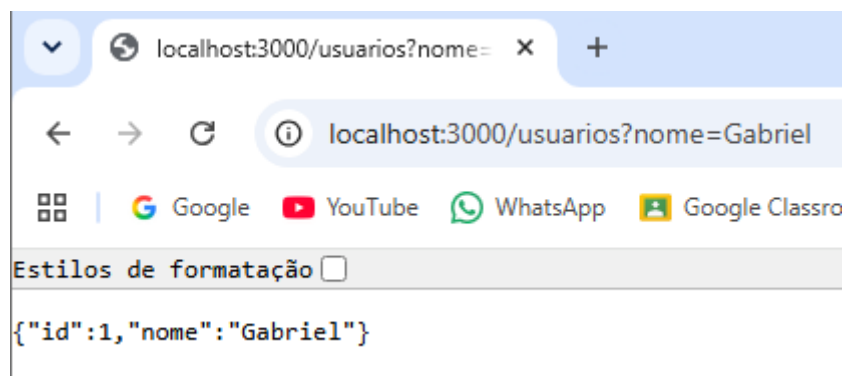
(Faltou :)

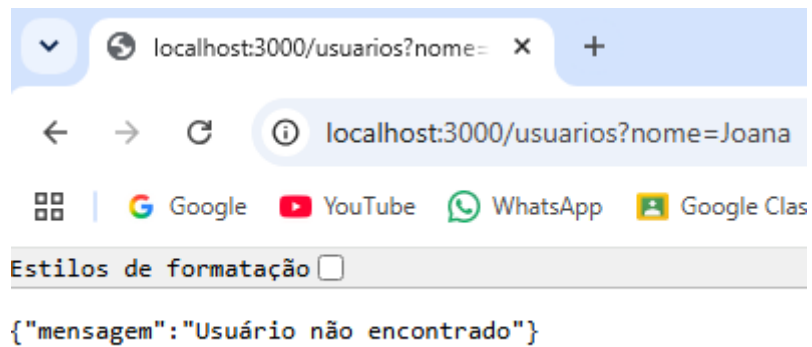


- Nesse caso, req.query será o termo após o símbolo ?;

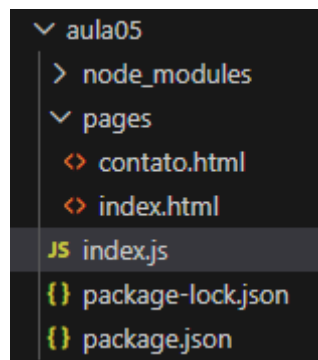
- Exemplo:

```
JS index.js X
aula05 > JS index.js > app.get("/usuarios") callback
9      PUT - Atualizar
10     DELETE - Remover
11  */
12
13  app.get("/", (req, res) => {
14    res.send("<h1>Bem-vindo!</h1>");
15  });
16
17  app.get("/usuarios", (req, res) => {
18    const { nome } = req.query;
19
20    const usuarios = [
21      { id: 1, nome: "Gabriel" },
22      { id: 2, nome: "João" },
23      { id: 3, nome: "Maria" },
24    ];
25
26    if (!nome) {
27      return res.json(usuarios);
28    }
29
30    const buscaUsuario = usuarios.find(usuario => usuario.nome === nome);
31    if (!buscaUsuario) {
32      return res.status(404).json({ mensagem: "Usuário não encontrado" });
33    }
34    return res.json(buscaUsuario);
35  });
36
37  app.listen(port, () => {
38    console.log(`Example app listening on port ${port}`);
39  });
```





- Até agora, nossas rotas retornavam textos ou JSON. Vamos ver como ligar uma página HTML a uma rota do Express:



```
JS index.js X
aula05 > JS index.js > ...
1  import express from "express";
2  import path from "path";
3  import { fileURLToPath } from "url";
4
5  const app = express()
6  const port = 3000;
7
8  const __filename = fileURLToPath(import.meta.url);
9  const __dirname = path.dirname(__filename);
10
```

- Importando os módulos:
 - Importa o framework Express, responsável por criar o servidor e as rotas;
 - O módulo path pertence ao próprio Node.js e serve para trabalhar com caminhos de arquivos;

- `fileURLToPath`: essa função será utilizada para obter o caminho do arquivo atual (necessária pois estamos usando `type: module`);
- `__filename`: caminho do arquivo atual;
- `__dirname`: caminho do diretório atual;

```

11  /*
12  |   GET - Buscar
13  |   POST - Criar
14  |   PUT - Atualizar
15  |   DELETE - Remover
16  */
17
18  app.get("/", (req, res) => {
19
20      res.sendFile(
21          path.join(__dirname, "pages", "index.html")
22      );
23
24  });
25

```

- Primeira rota:
 - Quando o navegador fizer uma requisição GET para a página inicial, a função será executada;
 - `res.sendFile`: envia um arquivo para o navegador;
 - `path.join`: combina as string para produzir o caminho completo do arquivo HTML;

index.html X

aula05 > pages > index.html > html > body > h1

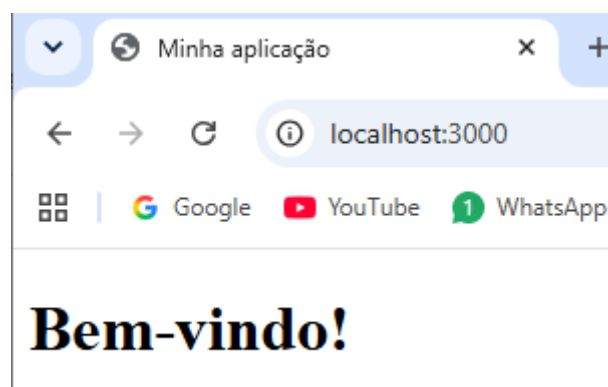
```

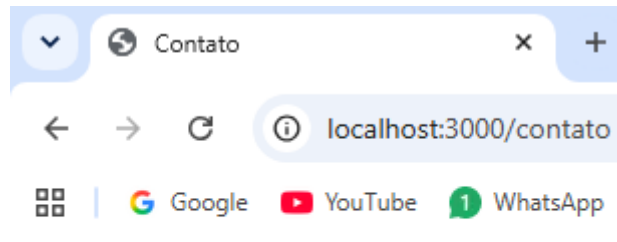
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Minha aplicação</title>
7  </head>
8  <body>
9      <h1>Bem-vindo!</h1>
10 </body>
11 </html>

```

```
contato.html X
aula05 > pages > contato.html > html > body > h1
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Contato</title>
7 </head>
8 <body>
9   <h1>Página de Contato</h1>
10 </body>
11 </html>
```

```
18 app.get("/", (req, res) => {
19
20   res.sendFile(
21     path.join(__dirname, "pages", "index.html")
22   );
23
24 });
25
26 app.get("/contato", (req, res) => {
27
28   res.sendFile(
29     path.join(__dirname, "pages", "contato.html")
30   );
31
32 });
```



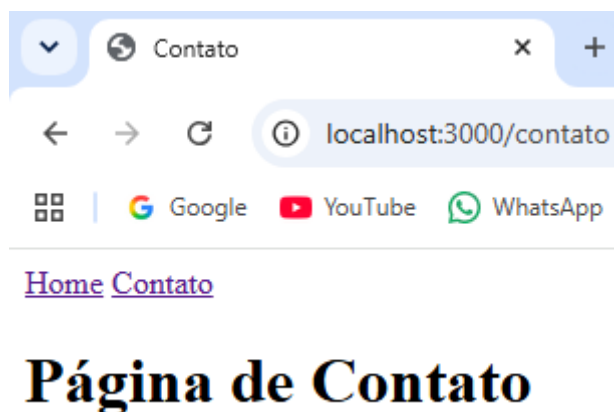
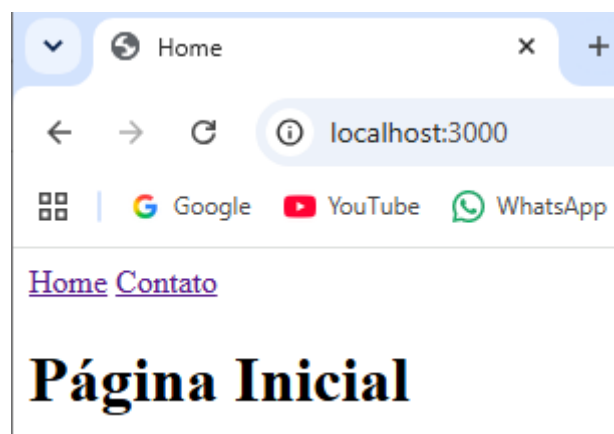


Página de Contato

- Com isso, podemos criar uma navegação dentro do HTML utilizando links:

```
index.html X
aula05 > pages > index.html > html > body > h1
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Home</title>
7  </head>
8  <body>
9    <nav>
10     <a href="/">Home</a>
11     <a href="/contato">Contato</a>
12   </nav>
13
14   <h1>Página Inicial</h1>
15 </body>
16 </html>
```

```
contato.html X
aula05 > pages > contato.html > html > body > h1
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Contato</title>
7  </head>
8  <body>
9      <nav>
10         <a href="/">Home</a>
11         <a href="/contato">Contato</a>
12     </nav>
13
14     <h1>Página de Contato</h1>
15 </body>
16 </html>
```



ATIVIDADE

ATIVIDADE - Catálogo de Filmes

- Garantir que todos tenham o Node.js instalado;
- Link: <https://nodejs.org/pt-br/download>.

Obs.: O tema de “filmes” é apenas uma sugestão para praticar os conceitos vistos em aula. Caso desejem, vocês podem escolher outro tema de interesse, como séries, livros, jogos, bandas, animes, produtos, esportes etc.

Objetivo: Desenvolver uma aplicação simples utilizando Node.js e Express.js, aplicando os principais conceitos vistos na aula:

- Criação de projetos Node.js;
- Configuração do Express;
- Criação de rotas GET;
- Envio de páginas HTML com `sendFile`;
- Criação de uma API simples com `res.json`;
- Utilização de parâmetros de rota (`req.params`); **(bônus)**
- Utilização de query strings (`req.query`); **(bônus)**
- Navegação entre páginas através de links.

O site deverá conter:

- Página Inicial (rota “/”): nome do site, pequena descrição do projeto, barra de navegação;
- Página de Filmes (rota “/filmes”): lista contendo alguns filmes favoritos;
- Página de Contato (rota “/contato”): nome do desenvolvedor, e-mail fictício, informações adicionais;
- Todas as páginas deverão possuir uma navbar para permitir a navegação entre elas.

Além das páginas HTML, o projeto deverá possuir uma rota da API (“/api/filmes”), que deverá retornar um array de objetos em formato JSON.

Bônus: Criar uma rota capaz de retornar informações de um filme específico e uma rota para simular uma pesquisa. (O foco é sempre a prática).